

MySQL ユーザ管理

アカウントはCREATE USERで作成します

次の例はアカウントhogeをパスワードabcdefgで作成する例です

```
mysql> CREATE USER hoge IDENTIFIED BY 'abcdefg';
```

作成したばかりのアカウントは何の権限も持っていません

MySQLサーバに接続することはできますが、それ以外のことは何もできません

アカウントに権限を与えるにはGRANTを使用します

次の例は、アカウントhogeに対して、データベースworkbookのすべてのテーブルについてSELECTできる権限を与えています

```
mysql> GRANT SELECT ON workbook.* TO hoge;
```

権限をはく奪するにはREVOKEを使用します

次の例は、アカウントhogeから、データベースworkbookのすべてのテーブルについてSELECTできる権限をはく奪します

```
mysql> REVOKE SELECT ON workbook.* FROM hoge;
```

アカウントを削除するにはDROP USERを使用します

```
mysql> DROP USER hoge;
```

基本的なユーザ管理のコマンドはこのような感じです

以降では、MySQLでのユーザ管理について細かい部分を紹介します

【認証について】

MySQLに接続するにはユーザ名とパスワードが必要です

アカウントを作成するときに**IDENTIFIED BY**でパスワードを指定しますが、パスワードを指定しない場合は、パスワードなしで接続できるアカウントが作成されるので注意してください

パスワードなしでのデータベースサーバ運用はとても危険です

また、今まで授業で使用してきたアカウント名『root』パスワード『root』も

望ましくないものですので、セキュリティを考慮したものを使うよう心掛けましょう

MySQLの認証方式はいくつかあり

プラグインなどを利用して認証方式を変更することもできるようになっています

デフォルトの認証方式は**cached_sha2_password**です

cached_sha2_password

MySQL8.0でのデフォルトの認証方式です

パスワードのハッシュ化にSHA256を使用していて、高速化のためにデータベースサーバのメモリにキャッシュを行います

デフォルトの認証方式なので、何も指定しなければこれが使用されますが

明示的に指定したい場合はアカウント作成時に下記のように指定します

```
mysql> CREATE USER hoge IDENTIFIED WITH cached_sha2_password BY 'abcdefg';
```

mysql_native_password

mysql_native_passwordはMySQL5.7までのデフォルト認証方式でした

古いクライアントから接続する必要があるなど、cached_sha2_passwordが使用できない場合にはこれを指定するといいかもしれません

```
mysql> CREATE USER hoge IDENTIFIED WITH mysql_native_password BY 'abcdefg';
```

こちらもMySQL8にもデフォルトで組み込まれているため、プラグインのインストールなどは不要となっています

授業で作成した『root』アカウントはこの認証方式を選択しています

↓確認コマンド

```
mysql> SELECT user, host, plugin FROM mysql.user;
```

その他プラグインには

OSのユーザ名と同じ名前のMySQLアカウントでパスワードなしで接続できる

auth_socketなどがあります

【パスワードの有効期限】

アカウントのパスワードに有効期限を設定することができます
有効期限はdefault_password_lifetime変数で日数を指定します
デフォルトは0で無期限です

ALTER USERを使用すれば、アカウントごとに有効期限を設定することもできます

(例) hogeアカウントのパスワードの有効期限を90日に設定

```
mysql> ALTER USER hoge PASSWORD EXPIRE INTERVAL 90 DAY;
```

(例) hogeアカウントのパスワードの有効期限を無期限に設定

```
mysql> ALTER USER hoge PASSWORD EXPIRE NEVER;
```

(例) 強制的にパスワードを期限切れにする

```
mysql> ALTER USER hoge PASSWORD EXPIRE;
```

パスワードが期限切れになった場合、古いパスワードで接続することはできませんが、
パスワードの変更を促され、それ以外の操作はできないようになっています

定期的にパスワードを更新してセキュリティを高めたいときに使用します

その他にも、同じパスワードを繰り返し再設定させないための仕組みや
パスワードの強度を確保するためのルールを設定することができる機能が用意されています
興味があれば調べてみてください

【権限】

アカウントの権限を確認するには**SHOW GRANTS**を使用します

```
mysql> SHOW GRANTS;
+-----+
| Grants for hoge@% |
+-----+
| GRANT USAGE ON *.* TO `hoge`@`%` |
| GRANT SELECT ON `workbook`.* TO `hoge`@`%` |
+-----+
2 rows in set (0.00 sec)
```

USAGEは接続できるだけの権限です

SELECTはデータベースに対して**SELECT**命令を実行できる権限です

権限の種類は多数あり、アカウントには必要な権限のみ割り当てることが望ましいです

権限	使用可能な機能
ALL [PRIVILEGES]	すべての機能 (PRIVILEGES は省略可)
ALTER	ALTER TABLE
ALTER ROUTINE	ALTER PROCEDURE / ALTER FUNCTION
CREATE	CREATE DATABASE / CREATE TABLE
CREATE ROLE	CREATE ROLE
CREATE ROUTINE	CREATE PROCEDURE / CREATE FUNCTION
CREATE TABLESPACE	CREATE TABLESPACE
CREATE TEMPORARY TABLES	CREATE TEMPORARY TABLE
CREATE USER	ALTER USER / CREATE ROLE / CREATE USER / DROP ROLE / DROP USER / RENAME USER / REVOKE ALL
CREATE VIEW	CREATE VIEW
DELETE	DELETE
DROP	DROP DATABASE / DROP TABLE / DROP VIEW / TRUNCATE TABLE

権限	使用可能な機能
DROP ROLE	DROP ROLE
EVENT	CREATE EVENT / DROP EVENT / ALTER EVENT
EXECUTE	ストアドプロシージャ / ストアド関数の実行
FILE	LOAD DATA / SELECT INTO OUTFILE / LOAD_FILE()
GRANT OPTION	GRANT / REVOKE (自身と同じ権限)
INDEX	インデックスの削除と破棄
INSERT	INSERT / ANALYZE TABLE / OPTIMIZE TABLE
LOCK TABLES	LOCK TABLES
PROCESS	SHOW PROCESS / SHOW ENGINE
PROXY	プロキシアカウントの作成
REFERENCES	外部キーの作成
RELOAD	FLUSH / RESET MASTER / RESET SLAVE
REPLICATION CLIENT	SHOW MASTER STATUS / SHOW SLAVE STATUS / SHOW BINARY LOGS
REPLICATION SLAVE	SHOW SLAVE HOSTS / SHOW RELAYLOG EVENTS / SHOW BINLOG EVENTS
SELECT	SELECT / EXPLAIN
SHOW DATABASES	SHOW DATABASES (全データベースに対する)
SHOW VIEW	SHOW CREATE VIEW / EXPLAIN
SHUTDOWN	SHUTDOWN / RESTART
SUPER	システム管理に関わる多くの命令
TRIGGER	CREATE TRIGGER / DROP TRIGGER / SHOW CREATE TRIGGER
UPDATE	UPDATE
USAGE	-

権限には沢山種類があります

アカウントを作成するたびにひとつずつ指定しては大変です...

用途に応じた複数の権限をまとめてアカウントに付与できれば便利そうです

次の例では、workbookデータベースのすべてのテーブルに対するSELECT権限と、特定のテーブルに対するINSERT権限を持つapp_userと workbookデータベース内のすべての権限をもつapp_devというロールを作っています

```
mysql> CREATE ROLE app_user;
```

```
mysql> GRANT SELECT ON workbook.* TO app_user;
```

```
mysql> GRANT INSERT ON workbook.books TO app_user;
```

```
mysql> CREATE ROLE app_dev;
```

```
mysql> GRANT ALL ON workbook.* TO app_dev;
```

作成したロールをアカウントに付与するには**GRANT** ロール **TO** アカウントを使用します

```
mysql> GRANT app_user TO user1;
```

※**GRANT**で権限が付与されているだけではその権限は有効になりません

SET ROLEを実行することでロールの権限が有効になります

逆に**SET ROLE NONE**を実行すればロールを使わない状態にすることもできます

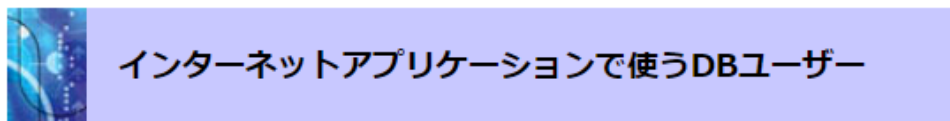
```
mysql> SET ROLE app_user;
```

【Webアプリケーションとデータベースアカウント】

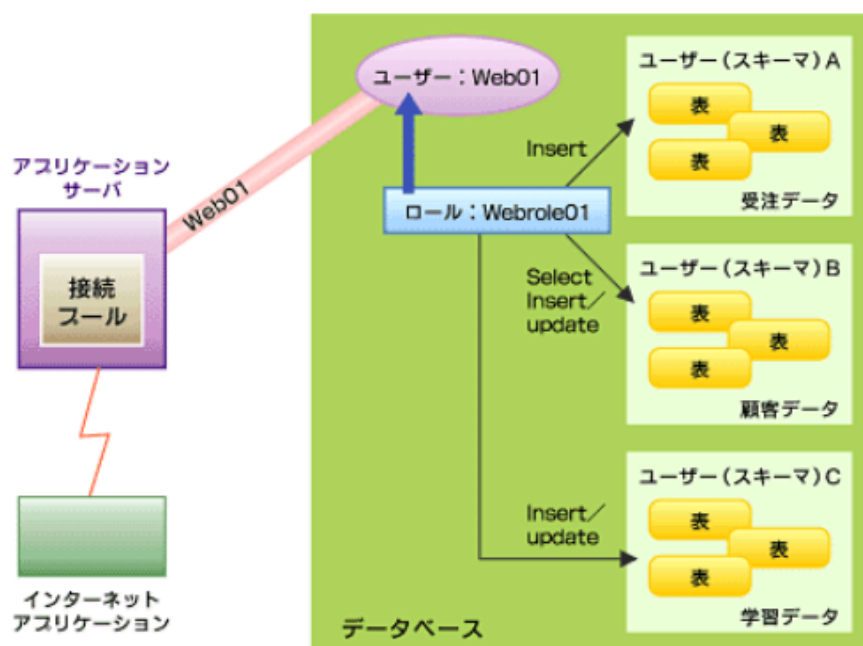
Webアプリケーションを作成する際には
Webアプリケーションのユーザアカウントごとにデータベースサーバのアカウントを作成することはあまりないです

よくあるのはWebアプリケーションのユーザで1つのデータベースアカウントを使いまわす
形です
その際には権限を適切に設定し、必要以上の操作をさせないように制限をかけるようにしましょう

以下、Webサイト転載



通常、DB側のユーザーと認証ユーザーを1対1にはしません。セキュリティ的な観点でいえば、使用するDBログイン名は1つでも構いません。つまり、アプリケーションサーバから接続してくる一般ユーザーは全員が1つのDBユーザーを利用しているという形です。



「なぜ分けないの？」と疑問に思うかもしれませんが、それはセキュリティ的に必要性がないからです。DBユーザーを分けても、結局はその接続認証はプログラム内でのハードコードになります（DBの認証情報を、一般ユーザーが知らせてあげるなんてしませんね。危ないだけです）。